

معماری پروژه مربع خوارزمی (DDD + Blazor WASM)

این پروژه با استفاده از معماری لایه‌ای ساده‌سازی شده بر اساس اصول DDD طراحی شده است.

۱. ساختار Solution و لایه‌ها

Khwarizmi.Domain (هسته مرکزی)

- مسئولیت: حاوی موجودیت‌ها (Entities)، اشیاء مقدار (Value Objects) و منطق خالص بازی.
- اجزا:
- Tile : موجودیت پایه برای قطعات با ویژگی‌های ابعاد و نوع هاشور.
- Puzzle : ریشه اصلی (Aggregate Root) مدیریت وضعیت یک مرحله و اعتبارسنجی چپ
- Player : موجودیت بازیکن شامل تاریخچه بازی‌ها (GameHistory) و سطح فعلی.
- DifficultyEvaluator : سرویس دامنه برای تعیین سختی پازل بعدی بر اساس تاریخچه

Khwarizmi.Application (لایه هماهنگ‌کننده)

- مسئولیت: مدیریت جریان بازی و تعامل بین UI و Domain.
- اجزا:
- GameManager : مدیریت چرخه حیات بازی (شروع پازل جدید، ثبت پیروزی).
- HistoryService : تحلیل تجربه بازیکن برای ارائه پازل‌های متناسب.
- DTOs : مدل‌های انتقال داده مانند GameStateDto و TilePositionDto .

Khwarizmi.Infrastructure (لایه زیرساخت)

- مسئولیت: ذخیره‌سازی داده‌ها در مرورگر.
- اجزا:
- BrowserStorageProvider : پیاده‌سازی ذخیره و بازیابی پروفایل بازیکن و تاریخچه بازی،
LocalStorage .

Khwarizmi.Presentation.Blazor (رابط کاربری)

- مسئولیت: رندرینگ بصری و تعاملات کاربر.
- اجزا:
- Pages/Game.razor : صفحه اصلی بازی شامل برد و لیست قطعات.

- Components/TileRenderer.razor : نمایش بصری قطعات با استفاده از CSS (هاشور Bootstrap)
- StateContainer : یک کلاس Service Simple برای مدیریت وضعیت (State) جاری بدو، از Fluxor

۲. نمای درختی پوشه‌بندی (Solution Tree)

```

KhwarizmiGame.sln
├── Core.1 ─┬──
│   ├── Khwarizmi.Domain ─┬──
│   │   ├── Entities // Puzzle, Player ─┬── (حاوی تاریخچه)
│   │   ├── ValueObjects // Dimensions, Coordinate, HatchPattern ─┬──
│   │   ├── Services // DifficultyCalculator ─┬── (منطق انتخاب پازل سخت‌تر)
│   │   └── Events // PuzzleSolvedEvent ─┬──
│   └── Khwarizmi.Application ─┬──
│       ├── Services // GameManager, HistoryProvider ─┬──
│       └── DTOs // PuzzleDto, PlayerProgressDto ─┬──
│
├── Infrastructure.2 ─┬──
│   ├── Khwarizmi.Infrastructure.Persistence ─┬──
│   └── LocalStorage // مدیریت ذخیره سازی در مرورگر ─┬──
│
└── Presentation.3 ─┬──
    ├── Khwarizmi.Client.Blazor ─┬──
    │   ├── Components // Board, Tile, PlayerStats ─┬──
    │   ├── Services // GameStateContainer ─┬── (مدیریت وضعیت محلی)
    │   ├── Pages // Game, History, About ─┬──
    │   └── wwwroot // CSS (Bootstrap + Custom Hatches) ─┬──
    
```

۳. تکنولوژی‌ها و انتخاب‌های فنی

- Framework: .NET 8 / .NET 9
- UI: Blazor WebAssembly
- Styling: Bootstrap 5 (برای چیدمان کلی و کامپوننت‌های UI)
- State Management: Simple Service State Pattern (استفاده از Dependency Injection روی داده‌های C# به جای Fluxor).
- Storage: LocalStorage (برای حفظ تاریخچه بازی‌های بازیکن منفرد).

۴. مکانیسم پیشرفت (Progression)

الگوریتم تولید پازل با لایه Application در ارتباط است تا:

1. تاریخچه بازی‌های موفق بازیکن را بررسی کند.

2. بر اساس تعداد پیروزی‌ها و سرعت حل پازل، پارامترهای n_i (تعداد مجهولات) و C (اعداد ثابت) ر $(ax + by + \dots + C)^2$ افزایش دهد.